From Short Programs to Complex Computation

An Empirical Study of Brainfuck Program Length, Runtime, and Output Productivity

Computational research report | Version 1.0 | 20 October 2026

Abstract

Short programs can execute many instructions without producing correspondingly large outputs or using much memory. We examine these distinctions through exhaustive enumeration of Brainfuck command strings of lengths one through six, with explicitly bounded execution. The primary model has wrapping 8-bit cells, a right-growing tape, a clamped left boundary, and empty input. Four saved configurations contain 897,608 candidate records, including invalid strings and repeated candidates across configurations. Increasing the instruction cutoff from 512 to 4,096 changes the observed maximum runtime at length six from 512 to 1,026 and maximum output from 6 to 510 bytes. At the higher cutoff, 4-bit and 8-bit cells yield identical per-program classifications through the compared lengths, despite different runtime and output records. Exact repeated-state certificates, selected witness replays, and a separate proof for four length-five growing-tape loops distinguish nontermination evidence from timeout. Eighty-eight length-six cases remain unresolved by the runner. The contribution is a reproducible, semantics-explicit empirical artifact and a study of censoring, rather than a new classical Busy Beaver value or an estimate of exact Kolmogorov complexity.

1. Introduction and Research Questions

Source length measures a description, while runtime measures a process. Neither alone determines how many output bytes a program emits, how far it moves through memory, or how many different final outcomes a collection of programs produces. Brainfuck offers a small command alphabet in which these quantities can be examined directly, including by inspecting individual witnesses.

Our starting point is an archived length-four experiment. Its 100-step budget left two programs unresolved. Extending the budget to 512 instructions resolves both as halting, while leaving the number of distinct outcomes unchanged. That example motivates a broader question: how much of an apparent relationship between length and productivity is a property of the language, and how much is created by the observation procedure?

We investigate three questions. **RQ1:** How do observed halting runtime records change with source length and execution cutoff? **RQ2:** How do output volume, visited memory and outcome diversity relate to those runtime records? **RQ3:** Which findings persist when the cell width changes from eight to four bits under otherwise identical rules?

The study is a finite census of candidate strings under selected settings. It is not a sample of human-written software, and it does not extrapolate a growth law beyond length six. “Productivity” here means measured execution, emitted bytes and memory visitation; it does not imply useful work, semantic richness, or information content.

2. Theoretical Background and Related Work

Busy Beaver. Radó introduced a finite-machine extremal problem with noncomputable behavior [1]. In the runtime formulation described by Aaronson [2], the maximum is taken over machines of a given state count that eventually halt on a blank tape. Our optimization instead fixes Brainfuck source length and a specific interpreter. Turing-machine state counts, Brainfuck command counts and their execution steps are different units. We therefore do not label our records as classical Busy Beaver values.

Language semantics. Cristofani's reference describes the eight commands and documents choices that vary across implementations [3]. Our loop jump follows its convention of continuing after the matching opening bracket. Wrapping arithmetic, the left clamp and EOF-zero are explicit model choices. The model identifier is part of every result; the language name alone is insufficient to reproduce a measurement.

Direct Brainfuck precedents. Čibej, Robič and Mihelič study counting and uniform generation of Brainfuck programs and sample runtimes under altered loop semantics [4]. Their six-command grammar differs from our eight-command candidate universe, which includes empty loops. Eterevsky's Beaver repository searches for long-running and high-memory programs using invariant reasoning [5]. Its documented dialect excludes I/O and terminates on arithmetic or left-boundary overflow. Those rules exclude our principal wrapping-cell mechanism, so a numerical ranking of the two projects' records would be misleading. Faase studies short programs producing numeric constants under specified final-state conditions [8]; numeric magnitude differs from our output-byte objective.

Cutoffs and complexity. Calude and Stay analyze halting probabilistically under assumptions about a distribution over time [6]. Such results do not turn this experiment's finite cutoff into a proof for an individual unresolved program. Soler-Toscano and colleagues use finite Turing-machine output frequencies in a method for approximating algorithmic complexity [7]. Our conditional frequencies over observed halting Brainfuck programs are not that method and are not exact Kolmogorov complexity.

For a fixed interpreter, an n-command program can be encoded using 3n bits because the alphabet has eight symbols. Together with a fixed decoder, this supplies a plain-description upper bound of 3n + c for its computable outcome. A prefix-free description additionally needs an appropriate self-delimiting encoding. Neither bound gives an exact shortest description. Likewise, the quantity -log2 f(x), with f normalized over observed halted outcomes, is conditional surprisal, not a complexity theorem.

Position of this study. Enumeration, semantic sensitivity and extremal Brainfuck searches are established topics. Our contribution is the integration of explicit rules, paired runs, raw records, witnesses and retained uncertainty in a small reproducible study. The literature search supports this modest positioning; it does not establish priority for any numerical record. The accompanying literature audit records sources and comparison limits.

3. Methodology

Let A be the eight-command alphabet and P(n) the balanced-bracket strings in A^n. We enumerate every raw string, retaining invalid syntax as a separate class. For cell width b and instruction budget B, let H(n,b,B) contain the programs observed to halt. For a halted program p, write t(p) for executed instructions, o(p) for output bytes, and m(p) for visited cells. Define:

```
T(n,b,B) = max { t(p) : p in H(n,b,B) }
O(n,b,B) = max { o(p) : p in H(n,b,B) }
M(n,b,B) = max { m(p) : p in H(n,b,B) }
D(n,b,B) = number of distinct outcomes among H(n,b,B)
U(n,b,B) = number stopped at a step or resource limit
```

These sets are nonempty in all reported runs. An outcome is the tuple (output bytes, final pointer, entire allocated tape), including trailing zero cells. Thus equal emitted text alone does not establish outcome equivalence. Visitation includes the initial cell and equals allocated tape length in this contiguous, right-growing implementation. It differs from final nonzero-cell count and host process memory.

The operational model initializes a single zero cell, grows rightward, clamps attempts to move left of zero, wraps arithmetic modulo 2^b, and writes zero at EOF. Each executed command costs one step. A nonzero closing bracket jumps after its matching opening bracket. Halting is recognized before applying the cutoff. Exact repeated states use (instruction pointer, data pointer, allocated tape, input position). Accumulated output is excluded because it cannot affect future execution. A repetition therefore proves nontermination under deterministic fixed-input semantics.

```
for each length n and each raw string p in A^n:
    if brackets are invalid: save invalid record; continue
    initialize machine and empty state history
    repeat:
        if instruction pointer is past source: classify halted; break
        if step budget is exhausted: classify step_limit; break
        check wall-time budget
        if control state repeats: classify cycle_detected; break
        check retained-history, tape and output budgets
        remember control state and execute one instruction
    save record; update classification counts and halted metrics
```

4. Experimental Setup

The primary run uses b=8, B=512, n=1..6. Two comparisons use B=4,096, n=4..6, with b=8 and b=4. All runs use empty input, a 60-second run budget, no candidate-count cap, 4,096 tape cells, 65,536 output bytes, 100,000 retained states and outcomes, and separate 16 MiB state/outcome payload caps. Every candidate was evaluated; no resource stop occurred. The model's right tape is unbounded; execution caps limit observation.

Engine 0.3.0 ran on Python 3.13.1, Windows 11, AMD64; manifests identify source commit 8a7ce03 and the OS-reported processor. Streaming exports retain compressed JSONL and CSV, configurations and hashes. Exact syntax counts provide an independent enumeration check: sum over k=0..floor(n/2) of C(n,2k) Catalan(k) 6^(n-2k), giving 6, 37, 234, 1,514, 9,996 and 67,181 valid strings. It counts bracket placements, balanced bracket shapes and six non-bracket choices.

5. Results and Visualizations

5.1 Runtime records depend on the cutoff

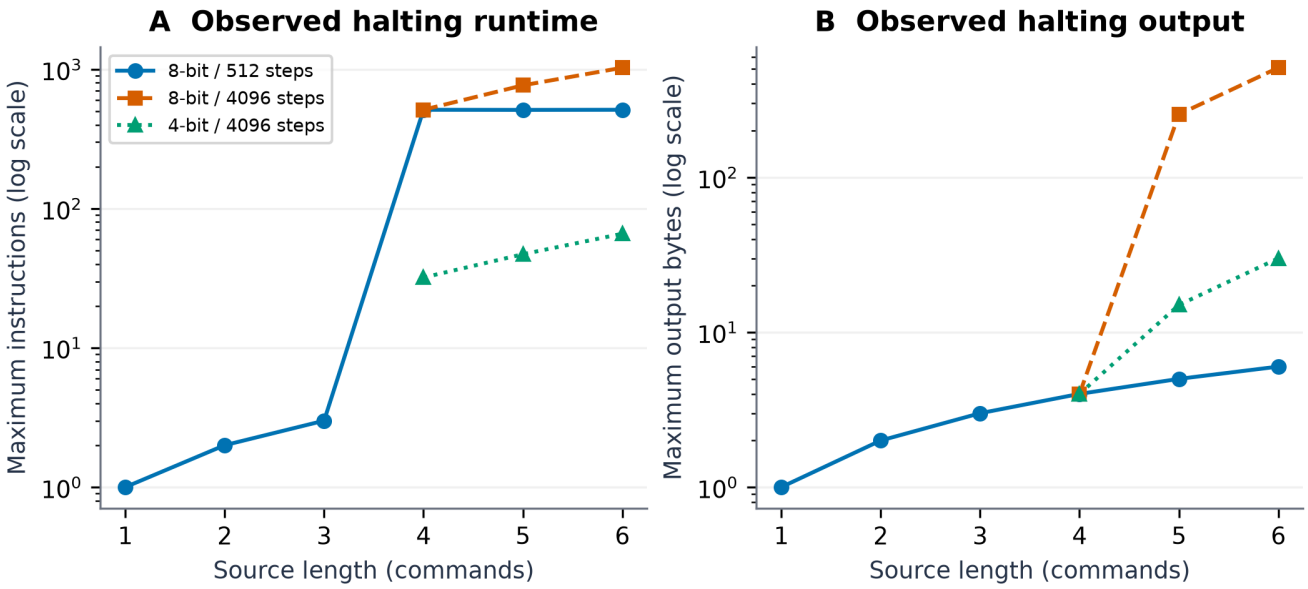


Figure 1. Observed maximum halting runtime and output by length and configuration. Logarithmic vertical axes; lines connect measured discrete lengths only. Overlapping points can obscure equal values. Incomplete classifications make the reported records lower bounds on unrestricted halting maxima.

At 512 steps, the observed runtime record decreases from 512 at length four to 511 at length five. This is censoring: the length-five program `+[+].` needs 513 steps and is excluded from the halted set. Raising the cutoff reveals records of 767 and 1,026 instructions at lengths five and six. Figure 1 also shows the much larger outputs missed by the lower cutoff.

n	Valid	Halted	Cycles	U	T	O	M	D
1	6	6	0	0	1	1	2	5
2	37	37	0	0	2	2	3	19
3	234	232	2	0	3	3	4	67
4	1514	1484	30	0	512	4	5	228
5	9996	9670	322	4	767	255	6	764
6	67181	64077	3016	88	1026	510	7	2536

Table 1. Eight-bit census: B=512 for n=1..3 and B=4,096 for n=4..6. Cycles are repeated-state proofs. U preserves raw runner classifications; the four length-five cases have separate nontermination proofs. Invalid strings are excluded here; raw candidate counts are 8^n. T, O and M concern halted executions only.

Of 36 length-five cases unresolved at 512 steps, 32 halt by 4,096. Of 480 length-six cases, 364 halt and 28 produce repeated-state proofs, leaving 88. These are paired program-level transitions. Exhaustive candidate coverage does not resolve the remaining execution uncertainty.

The archived 100-step length-four result is reproduced, not replaced: 1,482 halted, 30 cycles and two unresolved. Both additional halting programs at 512 steps reach already represented outcomes, preserving D=228.

5.2 Runtime and output measure different behavior

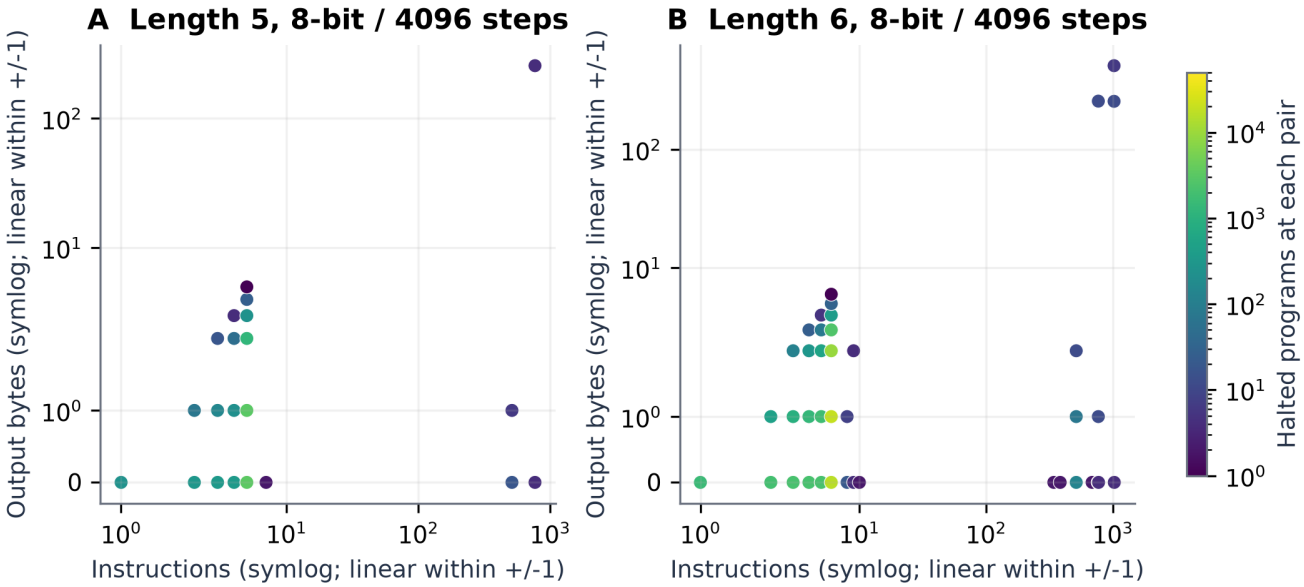


Figure 2. Complete observed halting runtime-output distributions at B=4,096. Each point aggregates all programs sharing that pair; color denotes the count on a logarithmic scale. Both axes are symmetric-log with a linear region within +/-1, retaining zero-output programs. Unresolved programs are excluded.

The distributions concentrate near short runtimes. At length five, 9,634 of 9,670 observed halted programs finish within five instructions (99.63%). At length six the corresponding count is 63,641 of 64,077 (99.32%). These are exact descriptive fractions of the observed halted sets, not estimates of eventual halting probability. Long-running programs matter to the maximum even when they contribute little to the population count.

Witness	n	8-bit steps	Output bytes
+[+]	4	512	0
+[<+]	5	767	0
+[+.]	5	767	255
+[<>+]	6	1026	0
+[+..]	6	1022	510

Table 2. Selected replayed witnesses. Ties exist; these are not unique maximizers. The length-six runtime and output records have different witnesses.

The length-five runtime witness +[<+] repeatedly increments its first cell; the clamped left move adds work. With b-bit cells its runtime is 2 + 3(2^b - 1). Replacing that move by output gives +[+.], with the same runtime and 2^b - 1 bytes. The length-six output witness emits twice per iteration: 2(2^b - 1) bytes in 2 + 4(2^b - 1) steps.

By contrast, +[<>+] enters a fresh second cell during its first iteration, then cycles that cell through its full range. Its runtime is 2 + 4*2^b. The first cell remains one, the second ends at zero, and no output is emitted. These elementary counts independently explain the records without treating emitted bytes as stored tape contents.

Eight-bit records yield 153.4 and 171 instructions per source command at lengths five and six, versus output records of 51 and 85 bytes per command. These ratios describe amplification of execution or output. They do not measure useful computation or compression performance.

5.3 Cell width, memory and outcome equivalence

At B=4,096, reducing cell width from eight to four bits changes runtime maxima at lengths four through six from 512, 767 and 1,026 to 32, 47 and 66. Output maxima change from 4, 255 and 510 to 4, 15 and 30. The formulas above explain these changes.

Nevertheless, matched program classifications agree at every compared length, with identical M, D and raw U totals. Runtime differs for 2, 34 and 398 programs halted in both models at lengths four, five and six. Equal outcome counts do not mean identical outcome values: arithmetic and emitted bytes can differ. The observed agreement is restricted to these two widths, three lengths and this cutoff.

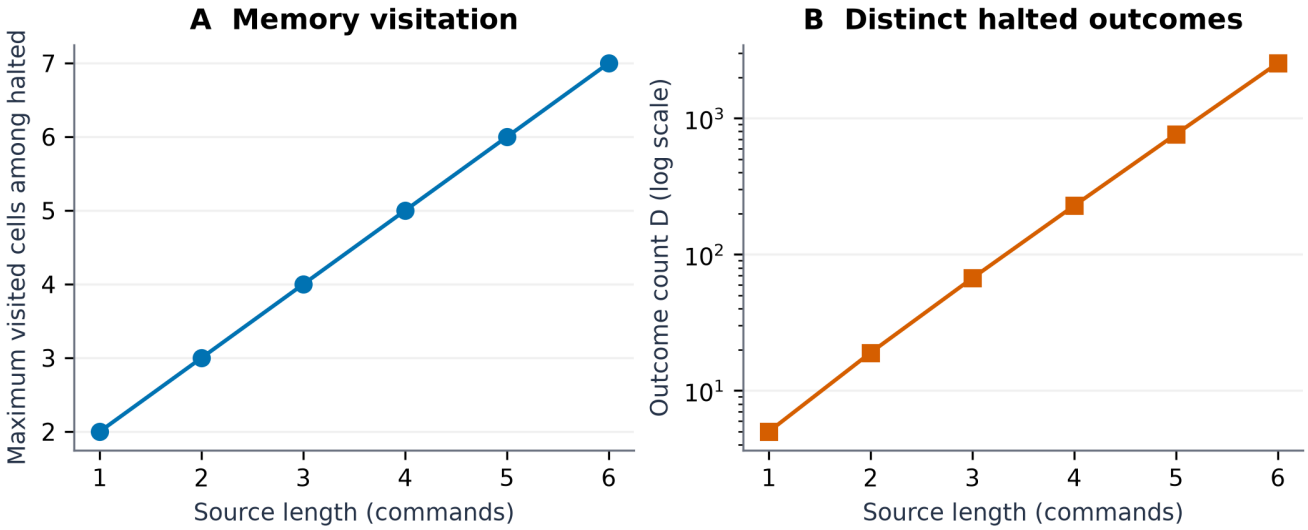


Figure 3. Eight-bit memory and outcome summaries, using B=512 for lengths 1-3 and B=4,096 for lengths 4-6, as in Table 1. Memory visitation includes the initial cell. Outcome identity includes output, final pointer and the full allocated tape. Values are observed halted summaries; length-six cases remain unresolved.

Maximum halted visitation is n+1 throughout the measured range, witnessed by n right moves. This is not a general memory bound for n-command programs: `+[>+]` visits unboundedly many cells without halting. Outcome diversity grows from 5 to 2,536, while the number of halted programs per distinct outcome grows from 1.2 to 25.27. The latter is an aggregate ratio, not a claim that outcome groups are equally frequent. Many sources can be equivalent under this experiment's outcome definition.

5.4 Evidence beyond timeout

All 10,078 cycle classifications across configurations were replayed, with equal control states recorded in certificates. The four length-five cases still step-limited at 4,096 are `+[>+]`, `+[>-]`, `-[>+]` and `-[>-]`. Each starts nonzero, moves right to a fresh zero cell, changes it to a nonzero value, and repeats. Induction proves nontermination in both tested models. Growing allocated tape prevents exact-state repetition. This argument supplements the preserved raw U=4; the 88 length-six cases have not received complete proof analysis.

6. Discussion and Potential Contributions

The main empirical result is the sensitivity of extrema to observation limits. High agreement in classifications or very short typical runtimes does not guarantee stable maxima. Paired semantic comparisons also show that matching class totals can conceal large quantitative differences. Reproducible witnesses make these effects inspectable. These observations support the artifact's educational and methodological value; neither the search strategy nor the underlying wrapping-loop mechanism is claimed as new research.

7. Limitations and Threats to Validity

Unresolved executions. The runner leaves 88 length-six cases unresolved at the higher cutoff. Consequently, length-six runtime, output, memory and diversity records are observed lower bounds. Length-five coverage plus the separate growing-tape proof resolves its residual cases at the model level, conditional on enumeration and implementation correctness. The raw datasets deliberately retain the original classifications.

External validity. Only lengths through six, empty input, two cell widths, one boundary rule and one outcome relation were studied. Four-bit cells are a sensitivity intervention. Full allocated tape distinguishes states that trimmed-tape or output-only equivalence could merge; these alternatives require recomputation.

Statistical interpretation. This finite census has execution censoring and potential implementation error, rather than sampling error. Sampling confidence intervals would not quantify either. No asymptotic growth or general monotonicity conclusion follows from the six lengths or their plotted connecting lines.

Validation scope. Integrity checks cover 897,608 records across configurations, including invalid syntax and repeated candidates. They compare encodings, hashes, order, syntax totals and summaries, without independently executing every record. All 33 selected witnesses were replayed; 24 eight-bit witnesses also match the archived interpreter. Four-bit record arithmetic has separate tests. A 37,448-record reproduction produced identical compressed records and scientific summaries. Shared implementation ancestry remains a risk; this is not formal verification.

Cost and reproducibility. Execution/export took about 2.38, 15.97, 19.06 and 18.27 seconds for the four configurations. These single-host observations exclude some finalization and verification. Length seven was deferred after an eightfold extrapolation of length-six cost suggested roughly two minutes, with uncertain tail costs. Cooperative time limits and payload caps are not hard CPU or process-RSS limits.

8. Future Research

A first extension is to analyze unresolved length-six programs with sound invariants and independently checkable evidence, preserving raw observations separately. Larger cutoffs and length seven should follow measured resource estimates, with incremental exports and explicit coverage.

A second direction is to compare outcome relations on saved runs, then vary EOF behavior, boundaries and inputs. Classification agreement across widths warrants testing on larger domains. An independently implemented interpreter would strengthen replication. The later website should expose settings, witness traces and uncertainty alongside charts.

9. Conclusion

Under the specified model, short Brainfuck programs amplify source length into substantially more execution and output, but these quantities are not interchangeable. Raising the cutoff reveals records hidden by censoring; changing cell width changes those records even when observed classifications agree. Explicit semantics, saved raw records and separate proof evidence make these conclusions auditable. The result is a bounded computational study with unresolved cases and a reusable research artifact, not a classical Busy Beaver solution or an exact complexity calculation.

References

[1] T. Radó. “On Non-Computable Functions.” *Bell System Technical Journal*, 41(3), 877-884, 1962. doi:10.1002/j.1538-7305.1962.tb00480.x.

[2] S. Aaronson. *The Busy Beaver Frontier*. Author-hosted survey, 2020. Full text.

[3] D. B. Cristofani. *Yet another brainfuck reference*. Undated language reference. Author's page.

[4] U. Čibej, B. Robič and J. Mihelič. *Empirical Estimation of the Halting Probabilities*. CiE 2014, author-uploaded conference manuscript. Full text. No unverified DOI or proceedings pagination is asserted.

[5] O. Eterevsky. *Beaver: a solver for halting problem of Brainfuck programs*. Software repository and README, undated. Original repository. Reported external results were not independently reproduced.

[6] C. S. Calude and M. A. Stay. *Most Programs Stop Quickly or Never Halt*. arXiv:cs/0610153v4, 26 January 2007. Verified preprint version.

[7] F. Soler-Toscano, H. Zenil, J.-P. Delahaye and N. Gauvrit. “Calculating Kolmogorov Complexity from the Output Frequency Distributions of Small Turing Machines.” *PLOS ONE*, 9(5), e96223, 2014. doi:10.1371/journal.pone.0096223.

[8] F. Faase. *Large numbers in BF*. Undated author-maintained page. Full text.

All web sources accessed 20 October 2026. Bibliographic records and a source-by-claim audit accompany the editable manuscript in `paper/references.bib` and `paper/LITERATURE_AUDIT.md`.

Data and Reproduction

The version-one research artifacts reside in `data/runs/v1/` and `data/analysis/v1/` in the project repository. Experimental configurations are in `experiments/configs/`; the engine provenance is commit 8a7ce03825edaac753f809a62c0ee5b7596bb9bb. The dataset release was committed as c63bed7cf31b5d81e46f0ea21565b9e88d05dd76.

`docs/PHASE_3_REPRODUCIBILITY.md` gives exact dataset creation, verification and reproduction commands. The paper's `README.md` gives isolated environment setup and figure/PDF build commands. Figures use the saved summary and complete aggregated runtime-output pairs; `statistics.json` records the descriptive fractions used here. No experiment was rerun solely to create a chart.

Artifact and Contribution Statement

This report was prepared with AI assistance for implementation, analysis and drafting. Numeric claims are grounded in versioned project data and the documented checks; the literature review is focused rather than systematic. No external peer review, independent replication, new record priority or practical compression benefit is claimed. The editable manuscript, build scripts, vector figures and verification record are supplied to enable further review.